

CPT111-2324 Lab 11 Pair Programming – List, Set, and Map

Aim

This week is focused on the List, Set, and Map. You will start using Pair Programming to learn more from each other.

One effective method for cultivating computational thinking skills while learning programming in computer labs is through Pair Programming. In this approach, two students collaborate on solving the same problem. One student assumes the role of the Driver, responsible for writing the code and translating design concepts into actual code implementation. Meanwhile, the other student, known as the Observer, reviews the code, identifies errors, and provides design suggestions and improvements. The roles of the driver and observer are continuously switched throughout the session.

在计算机实验室学习编程时，培养计算思维技能的一种有效方法是采用“结对编程”。在这种方法中，两名学生合作解决同一个问题。一名学生扮演“驾驶员”的角色，负责编写代码并将设计概念转化为实际的代码实现。与此同时，另一名学生，被称为“观察员”，他审查代码，识别错误，并提供设计、改进建议。在整个会话中，“驾驶员”和“观察员”的角色不断交替。

Pair Programming Lab:

1. The Lab TA will create a pair within the group.
2. Talk to your pair, who is going to be the Driver first.
3. Complete Task 1 with Pair Programming (30-45 minutes).
4. Show the TA the results of your works in Task 1.
5. Switch roles.
The Observer becomes the Driver and vice versa.
6. Complete Task 2 with Pair Programming (30-45 minutes).
7. Report to TA after you have completed Task 2.
8. TA gives grade.

Pair Programming Lab 11 Task 1: Using List with Wildcards

Wildcards (?) in generics provide flexibility when working with unknown types or when you need to work with a group of different types that share a common super type. In this task, you could use the format `List<? extends Animal>` to refer to any List with subclasses of Animal (e.g., `List<Cat>`, `List<Dog>`) to perform the same method differently. You can explore other usages of wildcards in `List` from online resources.

1. Create the Animal Class Hierarchy:
 - Create a hierarchy of classes where `Animal` is the base class and `Dog` and `Cat` are its subclasses. Each subclass (`Dog` and `Cat`) overrides the `makeSound()` method and may have additional methods unique to the subclass.
2. Create the Main Class with `main()` method:
 - Create a main class where you'll demonstrate the use of wildcards and generics with

the List.

3. Instantiate Lists and Work with Wildcards:
 - Inside the main method, create a List of Animals using a wildcard `<? extends Animal>` which allows the list to hold objects of type `Animal` or any of its subclasses (`Dog`, `Cat`, etc.).
4. Instantiate Specific Lists of Subtypes:
 - Create specific lists of `Dog` and `Cat` objects and add instances to them.
5. Assign Specific Lists to the Wildcard List:
 - Assign the specific lists (dogs, cats) to the wildcard list (animals). This works because of the covariance nature of Java generics. You can assign a `List<Dog>` firstly and then `List<Cat>` to a `List<? extends Animal>` by simply using “=” symbol.
6. Iterate Over the Wildcard List:
 - Iterate through the wildcard animals list and call the `makeSound()` method for each element. Since the list can hold `Animal` objects or its subclasses, you can call methods present in the `Animal` class.

Once you have written the code according to these steps, compile it and execute the main method. You should see the appropriate sound (Woof! or Meow!) being printed for each `Animal` object in the animals list, correspondingly.

Pair Programming Lab 11 Task 2: Customer Preferences Handling

Switch Driver $\rightleftharpoons$ Observer

Suppose you have an e-commerce platform where customers can select their favorite categories of products. You want to store their preferences efficiently and count how many customers have selected each category.

1. Create the `CustomerPreferences` Class:
 - Create a class named `CustomerPreferences` to handle customer preferences.
2. Implement `CustomerPreferences` Class:
 - **Instance Variables:** `customerPreferences` is a `Map` where the key is the `customer ID (String)` and the value is a `Set of preferences (String)`.
 - **Constructor:** Initialize `customerPreferences` in the constructor.
 - **`addCustomerPreference` Method:** Allows addition of customer preferences to the map.
 - **`countCategoryPreferences` Method:** Counts how many customers selected each category by iterating through preferences and updating a `Map` that tracks category counts.
 - **`Main()` Method:** Demonstrates the usage by simulating customer preferences and displaying the counts.
3. Simulate Customer Preferences:
 - Use the `addCustomerPreference` method to simulate customer preferences by adding customers and their preferred categories.
4. Count and Display Category Preferences:
 - Use the `countCategoryPreferences` method to count how many customers selected each category.

- Display the category counts to show the popularity of categories among customers.

Here is the example code for the addition of customer preferences:

```
preferences.addCustomerPreference("C1", new  
HashSet<>(Arrays.asList("Electronics", "Clothing", "Books")));
```

and the test cases could be:

```
preferences.addCustomerPreference("C1", new  
HashSet<>(Arrays.asList("Electronics", "Clothing", "Books")));  
    preferences.addCustomerPreference("C2", new  
HashSet<>(Arrays.asList("Electronics", "Beauty")));  
    preferences.addCustomerPreference("C3", new  
HashSet<>(Arrays.asList("Clothing", "Books", "Beauty", "Toys")));
```